

CHAPTER 6 OF 13 • PART FOUR

The Compute Kit

Choosing compute is choosing how much of the machine you still own. Statelessness is what makes a machine replaceable.



Thirteen chapters, gathered into six movements.

01 A Tuesday — one engineer, wake to sleep

02 The words — naming what you just watched

03 Protagonist and antagonist — where a design starts

04 Solution types — which answer is wanted

→ **Six kits — data, compute, network, scale, reliability, security**

06 Two designs, then the map back — Instagram, a parking app, and what you keep

WHERE THIS SITS

The data kit left a promise for this chapter to keep.

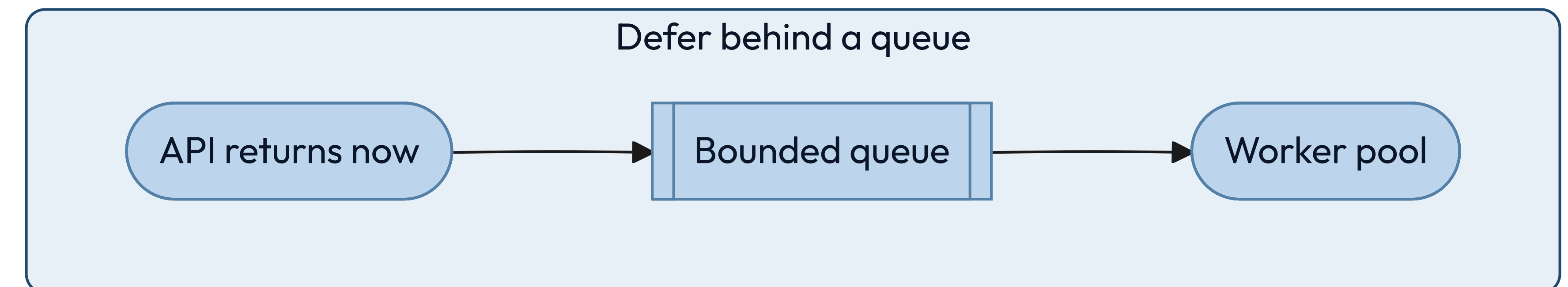
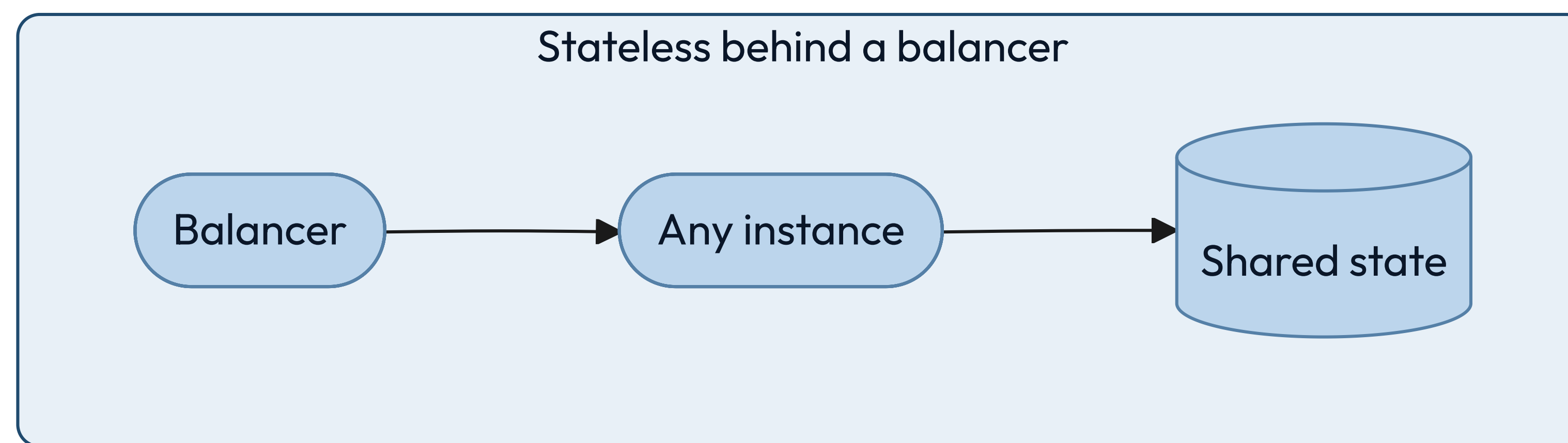
Every derived copy in chapter five — the index, the cache, the replica, the search store — rebuilds unattended. This chapter is what runs the rebuild. It assumes the three questions each kit entry answers, set out at the start of chapter five: what it is, when to reach for it, what it costs. Two of the four shapes decide whether the caller waits; the other two start from something that happened.

COMPUTE

Choosing compute is
choosing how much of
the machine you still own.

COMPUTE KIT • SOMEBODY IS WAITING

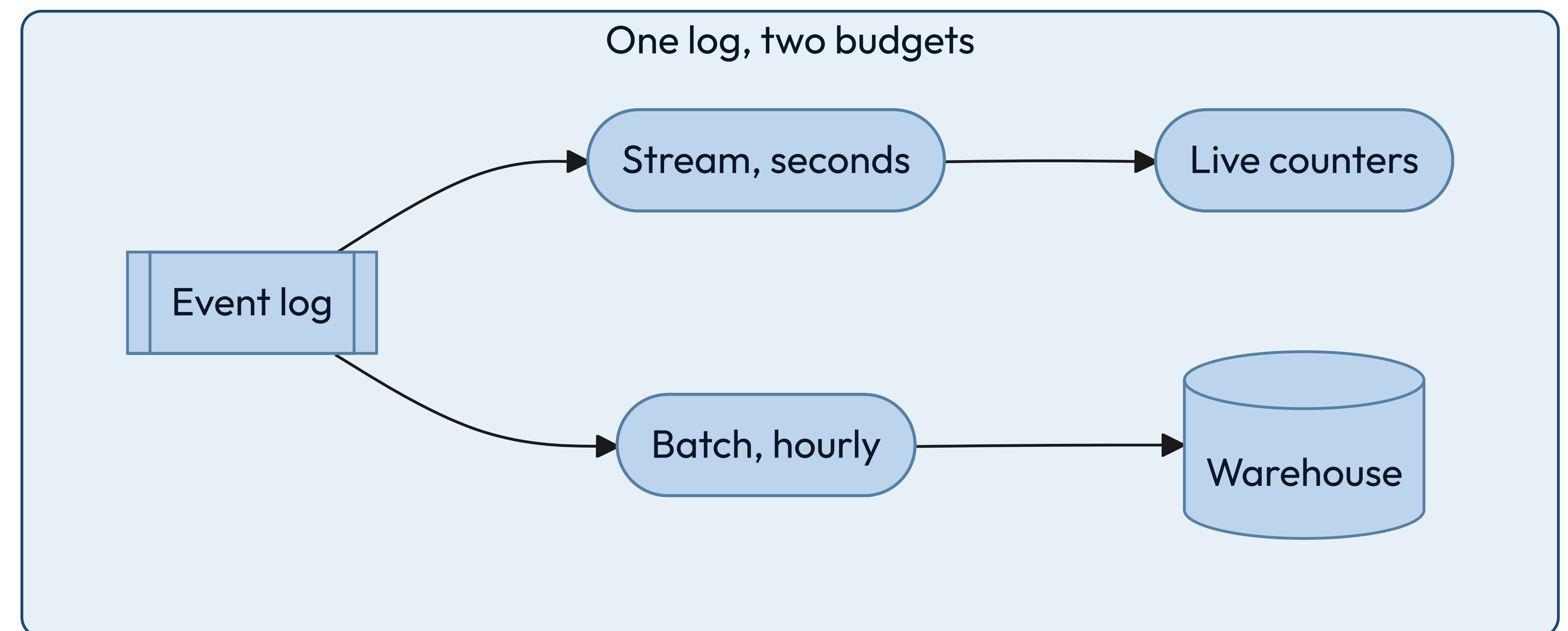
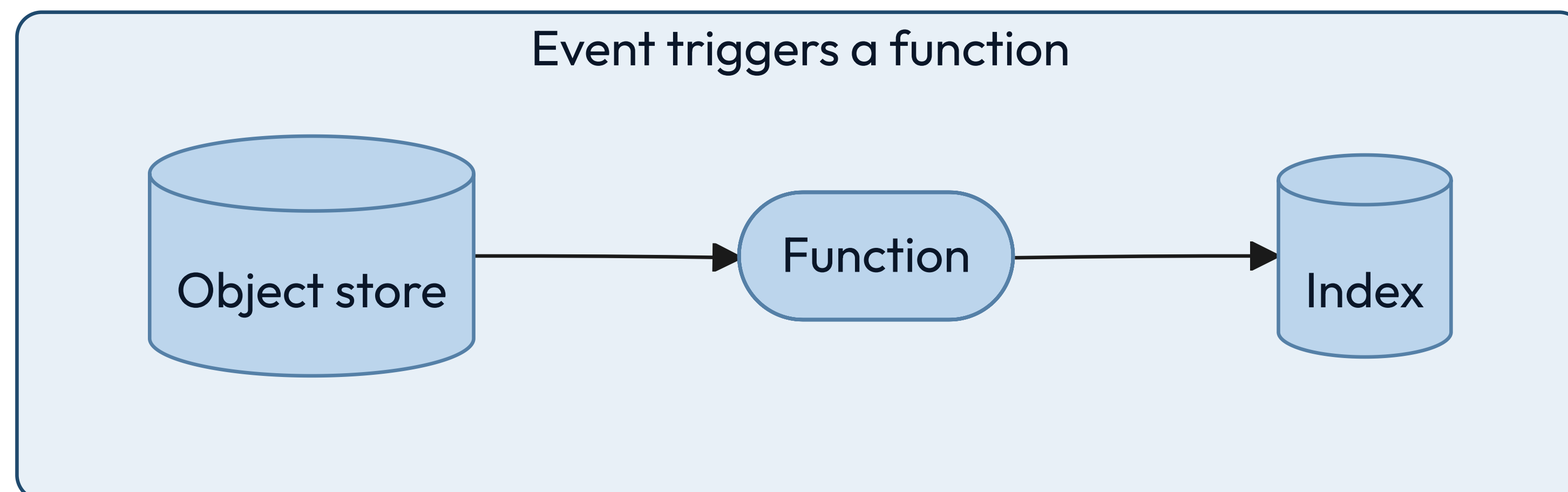
Two of the four shapes decide whether the caller waits for the work.

**KEY INSIGHT**

The balancer needs every instance to be interchangeable. The queue needs every worker to be repeatable.

★ The data kit closed owing two things it could not supply itself. A derived copy that rebuilds unattended needs something to run the rebuild, and an idempotent consumer needs something to be a consumer. Both are on this slide: the bounded queue and the worker pool that kit kept assuming.

The other two start from something that happened, not somebody who asked.



KEY INSIGHT

Nobody is on the phone, so the only deadline is one you promised somewhere else.

A virtual machine is what you reach for when the process outlives the request.

Reach for it when

The workload runs continuously, holds state in memory, or needs unusual kernel settings.

Walk away when

Traffic is spiky and idle machines start to dominate the bill.

The constraint you inherit

Patching, capacity planning, and the gap between staging and production are now yours.

A container makes the deployable unit identical everywhere it runs.

Reach for it when

You run many services, deploy often, and want one packaging story across all of them.

Walk away when

You run one small service. The scheduler that places containers on machines costs more than it saves at that size.

The constraint you inherit

The orchestrator is now infrastructure, with its own failure modes and its own pager.

A function is compute you rent by the millisecond, so idle costs you nothing.

Reach for it when

Traffic is spiky or rare, the work is short, and per-request isolation is welcome.

Walk away when

Runs are long, or steady traffic makes per-request pricing more expensive than a machine.

The constraint you inherit

Cold starts, unless you pay to keep instances warm — which is paying for idle again.

Statelessness is what makes a machine replaceable.

A STATEFUL INSTANCE

It holds something no other instance has: a session, a lock, a warm cache, an open connection. Scaling means moving that state, and a crash means losing it.



A STATELESS INSTANCE

Any instance serves any request because the state lives somewhere else. Scaling is arithmetic and failure is a routing change.

Anything you run should satisfy all four before it meets real traffic.

01

Any instance can be killed without a customer noticing

If that is false, you have state you have not named yet.

02

Deployments are reversible

A rollback is a routine operation, not an incident response.

03

Capacity is a number somebody owns

Not "it autoscales" — a ceiling, a cost, an owner. Maya's build had none of the three.

04

Startup does not depend on startup order

Services retry into each other instead of requiring a sequence.

YOUR TURN

Three workloads land on your desk. Say what each one runs on.

One: a nightly job that reads the whole orders table and writes one report file — forty minutes, once a day. Two: an API taking three thousand requests a second, deployed six times a day by four teams. Three: a thumbnail made whenever somebody uploads a photo — a few hundred a day, at no fixed time.

Name what each should run on, and the invariant it fails first if you get it wrong.

Then turn the page.

ONE ANSWER

Idle decides the last one. Neither of the first two is a cost argument.

01 The nightly report

A machine or a container on a schedule; what it is not is a function. Forty minutes outlives most function runtime caps, and one long run is not spiky. The invariant it fails first: capacity is a number somebody owns.

02 Three thousand a second, six deploys a day

Containers behind a balancer — many teams, frequent deploys, one packaging story. It fails "any instance can be killed" the moment somebody keeps a session in memory.

03 A few hundred thumbnails

A function on the upload event. Idle is most of the day and costs nothing, and the cold start it charges for is one nobody is waiting on. The invariant it fails first: startup does not depend on startup order — the event fires whenever it fires, so a function that assumes its index is already up breaks at 3am.

CHAPTER 6 OF 13 • HOW TO THINK ABOUT SYSTEMS

Your code runs somewhere now.

Chapter seven is the trip to it. Distance is the one cost you cannot optimize away, and most latency arguments end the moment somebody says the actual numbers.

Glossary

A

TERM	DEFINITION
API	The contract one system offers another: the operations and what they promise.